COMP206 learning journal

Hector Barquero

Jul 12, 2026

CONTENTS:

1 Submission instructions **1**
- 1.1 Rules for deliverables . . . 1
- 1.2 Program rules . . . 1
- 1.3 Program naming . . . 2
- 1.4 Program test plans . . . 2

2 Assignment instructions **3**
- 2.1 Assignment journal overview . . . 3
- 2.2 Technical requirements . . . 3
- 2.3 Test plan requirements . . . 4

3 Assignment one journal **5**
- 3.1 Overview and reflections . . . 5
- 3.2 Readings . . . 6
- 3.3 Practice problems . . . 16
- 3.4 Program design . . . 16
- 3.5 Sources and references . . . 16

4 Assignment two journal **17**

5 Assignment three journal **19**

6 Assignment four journal **21**

7 Practice problems **23**

8 Think C++ **25**
- 8.1 Chapter synopsis and notes . . . 25

9 Rooks guide to C++ **31**
- 9.1 Chapter synopsis and notes . . . 31

10 Programming fundamentals **37**
- 10.1 Synopsis . . . 37
- 10.2 Interesting notes . . . 37
- 10.3 Definitions . . . 37

CHAPTER

ONE

SUBMISSION INSTRUCTIONS

> **Tip**
>
> Refer to GoodDocs.cpp for a sample test plan format.

1.1 Rules for deliverables

Each assignment has multiple programs. All programs should be in a single directory. Don't nest them or make it harder to find. Remember that a human grades your work.

- Programs must run from command line
- Submit all source combined into one .zip or .gz.
- The .zip must unpack to a single working directory with all source files
- Only submit source .cpp, .h and Makefiles are optional. Include any input files. Submit only the source files. Do not submit compiled code.
- Upload through the **Assignment** tool
- Don't use .rar
- Don't submit compiled files (.obj or .exe)
- Don't submit project files generated by IDE

1.2 Program rules

1. Program must be complete, compile and run using the compiler and version specified for the course (the latest version of the GNU compiler). Failure to compile is a zero grade.
2. Program must solve problem in the assignment.
3. Site code used from the textbook or external source, if any.
4. Programs must have in-program documentation contained in comment block located at the start of the program. Name, AU student ID, and current date must be included. Include program name, program purpose in one sentence, and any special instructions to compile and execute the program. If it accepts or requires CLI arguments, it must be included in this in-program documentation.

1.3 Program naming

No matter what the program requirements, the main entry point for the program should be contained in a source file named according to the following the format:

Note

Assignment#Question#.cpp

1.4 Program test plans

A complete test plan documentation comment block must follow the main documentation comment block.

At a minimum, you must test:

1. Normal data: Standard input and outputs. Ensure accuracy.
2. Abnormal data: How does it handle invalid input, division by zero, input files not found. Test a variety of error conditions.
3. Limiting conditions: Zero, null, large and small numbers. Find the algorithmic limits.

CHAPTER

TWO

ASSIGNMENT INSTRUCTIONS

2.1 Assignment journal overview

> **Note**
>
> You can create a single journal for the whole course, or create a journal for each assignment.

1. Several pages long.
2. Cover the time you spent.
3. Document the difficulties you encountered, and how you overcame them.
4. Write down the activities you're doing ongoing to track reflections.
5. List the sources used.
6. Include the design and design decisions you made.
7. Match the learning outcomes to syllabus.

2.2 Technical requirements

1. Submit all files in a single .zip or .gz (no .rar, no proprietary compression).
2. Must unzip to one folder. Don't place each program into it's own working directory (./program1/, ./program2/).
3. Upload through the assignment tool.
4. Submit .cpp, .h. Make files are optional. Include input files.
5. Don't submit compiled files (.obj, .exe). No files generated by IDE.
6. Must execute from CLI.
7. Use the latest GNU C++ compiler.
8. In-program documentation required in a comment block at the top. See GoodDocs.cpp for an example. (Include your name, AU ID, date, program name, purpose in one sentence, instructions to run, and if it accepts CLI arguments).
9. After the comment block, include a test plan comment block. See GoodDocs.cpp.
10. Entry point for submissions are file Assignment#Question#.cpp, for example Assignment1Question1.cpp

2.3 Test plan requirements

1. Test normal data (standard inputs)
2. Test abnormal data (wrong char, types, file not found)
3. Test limit conditions (null, bytes exceed types, ttfb)

CHAPTER

THREE

ASSIGNMENT ONE JOURNAL

Time spent	Jan 2026 to Aug 2026
Student name	Hector Barquero

Tip

How to use this journal

This journal section acts as an index, with the reading contents contained in the **Readings** chapter. You can use this to jump to the correct chapter, and it organizes the Unit 1 assignment journal components more cleanly rather than having them duplicated.

3.1 Overview and reflections

3.1.1 Unit 1: Points to ponder

https://learning.athabascau.ca/d2l/le/18658/discussions/threads/337601/View?ou=18658

I've programmed in C, C++ a lot and reading these feels like a nice refresher.

I've been programming in javascript a lot lately, so I almost forget definitions of things and some of the differences feel jarring. I caught myself forgetting that C++ doesn't do hoisting, and I forgot how strict it is with type declaration… Reading this content again has me remembering I have some javascript habbits to break.

I also appreciate that both Think C++ and The Rooks Guide especially are really short and direct. I thought at first that they were quite dated, but I guess depending where you work… the C++ standard you might be writing could be dated too. (Where I am, I think it's an archaic standard, and the job previous to that we weren't using the latest either)

3.1.2 Unit 2: Points to ponder

Hector Barquero posted Feb 27, 2026 10:19 AM https://learning.athabascau.ca/d2l/le/18658/discussions/threads/337650/View

Completing unit 2 was interesting because it has differences to modern c++ that I needed to refresh on. I recall learning the same traditional academic declarations when first studying c++ at Fanshawe College in 2016:

but when I started working in 2018+, I used brace form since it's modern best practice with c++:

This might be another thing for me to remember since I'm used to protecting type narrowing when doing narrowing conversion, but I scanned ahead for other units and see that the text prefers explicit casting conversion.

This is all really good for me to read again because in javascript, everything is dynamically typed where the type belongs to the value, not the variables

3.2 Readings

3.2.1 Unit 1

ThinkCScpp 1.1: What is a programming language?

- A programming language provides formal rules for expressing instructions a computer can execute.
- High-level languages like C++ must be translated into machine language before they can run.

ThinkCScpp 1.2: What is a program?

- A program is a sequence of instructions that performs computation, input, output, testing, and repetition.

ThinkCScpp 1.3: What is debugging?

- Debugging is the process of finding and correcting errors in a program.
- Errors generally fall into compile-time, run-time, and logic categories.
- Debugging requires testing assumptions rather than randomly changing code.

ThinkCScpp 1.3.1: Compile-time errors

- Compile-time errors prevent the compiler from translating the source code.
- They commonly result from invalid syntax, missing punctuation, or incorrect declarations.

ThinkCScpp 1.3.2: Run-time errors

- Run-time errors occur after compilation while the program is executing.
- A program might crash, stop unexpectedly, or perform an invalid operation.
- The compiler can't always detect conditions that only appear during execution.
- Testing different inputs helps identify when the failure occurs.

ThinkCScpp 1.3.3: Logic errors and semantics

- Logic errors produce incorrect results even though the program compiles and runs.
- Semantic errors happen when valid code doesn't express the intended meaning.

ThinkCScpp 1.3.4: Experimental debugging

- Experimental debugging uses controlled tests to confirm or reject explanations for an error.
- Debug output can reveal variable values and show which statements execute.
- Changing several things at once makes it harder to identify the actual cause.

ProgFund 1: Introduction to Programming

- Programming converts a problem-solving process into instructions a computer can follow.
- Programs typically accept input, process data, and produce output.
- Good programming requires planning, implementation, testing, and maintenance.

ProgFund 1.1: Systems Development Life Cycle

- The SDLC organizes software work into planning, analysis, design, implementation, testing, and maintenance.
- Each phase reduces uncertainty before more development effort is committed.

ProgFund 1.3: Modularization and C++ Program Layout

- Modularization divides a large problem into smaller functions with specific responsibilities.
- A C++ program normally contains directives, declarations, functions, and a `main` function.
- Smaller modules are easier to test, reuse, and understand.
- Clear layout helps readers recognize the program's structure quickly.

ProgFund 2: Program Planning & Design

- Program planning defines the problem and solution before code is written.

ProgFund 2.1: Program Design

- Program design identifies inputs, processing steps, outputs, and required decisions.
- A planned solution is easier to implement and test than an improvised one.
- Complex problems should be divided into manageable modules.

ProgFund 2.2: Pseudocode

- Pseudocode describes program logic using structured language without strict C++ syntax.
- It helps refine algorithms before implementation details become a distraction.

ProgFund 2.3: Test Data

- Test data checks whether a program handles normal, boundary, and invalid inputs correctly.
- Expected results should be decided before the program is executed.
- One successful test can't prove that every possible input works.
- Carefully selected cases can expose incorrect conditions and assumptions.
- Tests should be repeated after changes to detect regressions.

ThinkCScpp 1.4: Formal and natural languages

- Natural languages tolerate ambiguity, while formal languages require precise syntax and structure.
- C++ can't infer intended meaning when its grammatical rules aren't followed.

ThinkCScpp 1.5: The first program

- A basic C++ program defines `main` as its starting point.
- `cout` writes output through the standard output stream.
- Statements generally end with semicolons.
- Compilation and execution are separate steps.

ProgFund 5: Integrated Development Environment

- An IDE combines tools for editing, compiling, running, and debugging programs.
- It simplifies development but doesn't replace understanding the build process.

ProgFund 5.1: Integrated Development Environment

- IDEs provide features such as syntax highlighting, project management, and error navigation.
- The compiler still determines whether C++ source code is valid.
- Debuggers let programmers inspect execution without adding permanent output statements.

ProgFund 5.2: Standard Input and Output

- `cin` reads values from standard input and `cout` writes values to standard output.
- Stream operators indicate whether data moves into or out of a stream.
- Input must match the destination variable's expected type.
- Output can combine text, values, and formatting within one statement.

ProgFund 5.3: Compiler Directives

- Preprocessor directives begin with # and are processed before compilation.
- `#include` makes declarations from headers available to the source file.

3.2.2 Unit 2

ThinkCScpp 2.1: More output

- Multiple values can be sent to `cout` using consecutive insertion operators.
- Escape sequences represent characters such as newlines and tabs.
- Output statements don't automatically add spacing between values.

ThinkCScpp 2.2: Values

- A value is a basic piece of data such as an integer, character, or string.
- Every value has a type that determines its meaning and permitted operations.

ThinkCScpp 2.3: Variables

- A variable is a named storage location associated with a specific type.
- Variables must be declared before they can be used.
- A variable's current value can change during execution.
- Using descriptive names makes expressions easier to understand.

ProgFund 3: Data & Operators

- Data types define how values are represented and which operations are valid.
- Operators combine or modify values to produce new results.

ProgFund 3.1: Data Types in C++

- C++ includes types for integers, floating-point numbers, characters, Boolean values, and text.
- Choosing an appropriate type affects memory use, precision, range, and available operations.
- C++ won't treat every type as interchangeable without a conversion.

ProgFund 3.2: Identifier Names

- Identifiers name variables, functions, types, and other program elements.
- An identifier can't begin with a digit or match a reserved keyword.
- Consistent naming conventions make related identifiers easier to recognize.

ProgFund 3.3: Constants and Variables

- Variables may change, while constants prevent reassignment after initialization.
- Constants communicate that a value isn't expected to change.
- `const` lets the compiler enforce that intention.
- Both constants and variables require suitable types.

ProgFund 3.4: Data Manipulation

- Expressions manipulate data through operators, variables, constants, and function calls.

ThinkCScpp 2.4: Assignment

- Assignment stores the right-hand value in the variable on the left.
- Assignment replaces the variable's previous value.
- The assignment operator doesn't mean mathematical equality.

ThinkCScpp 2.5: Outputting variables

- Variables can be inserted into an output stream alongside literal text.
- Output reflects the variable's value at the time the statement executes.

ThinkCScpp 2.6: Keywords

- Keywords have predefined meanings in C++ and can't be used as identifiers.
- Examples include `int`, `return`, `if`, and `while`.
- Editors often highlight keywords, but the compiler enforces their meaning.
- Keyword spelling and capitalization must be exact.

ProgFund 3.5: Assignment Operator

- The assignment operator evaluates the right-hand expression before storing its result.
- The left side must refer to a writable storage location.

ProgFund 3.6: Arithmetic Operators

- C++ supports addition, subtraction, multiplication, division, and remainder operations.
- The operand types affect how an arithmetic expression is evaluated.
- Division between integers discards the fractional part.
- Parentheses can make the intended grouping explicit.
- Invalid arithmetic, such as integer division by zero, can't produce a useful result.

ProgFund 3.7: Data Type Conversions

- Conversions change a value from one type to another.
- Implicit conversions happen automatically, while casts request a conversion explicitly.
- Narrowing conversions can lose range or precision.

ProgFund 4: Often Used Data Types

- Frequently used C++ types include integers, floating-point values, characters, strings, and Booleans.

ProgFund 4.1: Integer Data Type

- Integer types represent whole numbers without fractional components.
- Their supported range depends on the type and implementation.
- Signed types support negative values, while unsigned types don't.
- Overflow can produce incorrect or implementation-dependent behaviour.

ProgFund 4.2: Floating-Point Data Type

- Floating-point types represent values with fractional components and wide ranges.
- Many decimal fractions can't be represented exactly in binary.
- `double` generally offers more precision than `float`.

ProgFund 4.3: String Data Type

- `std::string` stores and manipulates sequences of characters.
- Strings support operations such as concatenation, comparison, indexing, and length checks.

RooksGuide 2: Variables

- Variables associate names with typed storage that a program can read and modify.
- C++ variables must be declared because their types are determined before execution.
- Initialization gives a variable its first value.
- Uninitialized local variables can contain indeterminate data.

RooksGuide 2.1: How do I decide which data type I need?

- Choose a type based on the required values, precision, operations, and range.
- The smallest possible type isn't always the clearest or safest choice.

RooksGuide 2.2: Identifiers

- Identifiers should describe their purpose without conflicting with C++ naming rules.
- Meaningful names reduce the need for explanatory comments.
- C++ identifiers are case-sensitive.

RooksGuide 2.3: Declaring a Variable

- A declaration introduces a variable's name and type to the compiler.

RooksGuide 2.4: Initializing Variables

- Initialization assigns a value when a variable is created.
- Brace initialization can reject some conversions that would lose information.
- Initializing variables early prevents accidental use of unknown values.

ThinkCScpp 2.7: Operators

- Operators perform computations using one or more operands.
- The same operator may behave differently with different types.
- Some operators modify values, while others only calculate results.
- Operator misuse can still compile when the expression is syntactically valid.

ThinkCScpp 2.8: Order of operations

- Operator precedence determines which parts of an expression are evaluated first.
- Parentheses should be used when the intended order isn't immediately clear.

ThinkCScpp 2.9: Operators for characters

- Character values can be compared and manipulated using their numeric encodings.
- Arithmetic on characters usually produces an integer result.
- Character ordering depends on the encoding used by the implementation.

ProgFund 4.4: Arithmetic Assignment Operators

- Operators such as += and *= combine arithmetic with assignment.
- `x += 2` is generally equivalent to `x = x + 2`.
- Compound assignment avoids repeating the variable name.
- The usual type conversion rules still apply.

ProgFund 4.5: Lvalue and Rvalue

- An lvalue identifies an object or storage location that may appear on an assignment's left side.
- An rvalue is commonly a temporary value or expression result.
- Assignment requires a modifiable lvalue as its destination.

ProgFund 4.6: Integer Division and Modulus

- Integer division removes the fractional portion of the result.
- The modulus operator returns the remainder from integer division.
- Modulus is useful for checking divisibility and extracting digits.
- Negative operands can make remainder results less intuitive.

RooksGuide 2.5: Assignment Statements

- Assignment changes an existing variable's value after it has been declared.
- The source expression is evaluated before the destination is updated.

RooksGuide 3: Literals and Constants

- Literals write fixed values directly in source code, while named constants give those values meaning.
- Constants prevent values from being changed accidentally.
- Replacing unexplained literals with named constants improves maintainability.

RooksGuide 3.1: Literals

- A literal represents a value directly, such as `42`, `3.14`, `'A'`, or `"text"`.
- Literal syntax helps determine the value's type.

RooksGuide 3.2: Declared Constants

- A declared constant associates an immutable value with a descriptive identifier.
- Constants should usually be initialized when declared.
- `const` allows the compiler to reject later assignments.
- Named constants avoid repeating unexplained values.
- Changing one declaration can update every expression that uses the constant.

RooksGuide 4: Assignments

- Assignment stores an expression's result in an existing object.
- Chained assignments are possible but can reduce readability.
- Assignment isn't interchangeable with initialization in every context.

ThinkCScpp 2.10: Composition

- Composition combines variables, literals, operators, and function calls into larger expressions.
- Complex expressions should remain readable and avoid repeating unnecessary calculations.

3.2.3 Unit 3

ThinkCScpp 3.1: Floating-point

- Floating-point values approximate real numbers using limited binary precision.
- `double` is commonly preferred when fractional calculations need reasonable precision.
- Equality comparisons can fail when rounding produces slightly different results.

ThinkCScpp 3.2: Converting from double to int

- Converting a `double` to `int` discards the fractional part rather than rounding it.
- Values outside the integer type's range can't be represented safely.
- An explicit cast makes the conversion visible to readers.
- Rounding functions should be used when truncation isn't intended.

ThinkCScpp 3.3: Math functions

- The <cmath> header provides functions for roots, powers, rounding, and trigonometry.
- Math functions accept arguments and return calculated results.

ThinkCScpp 3.4: Composition

- Function calls can be nested inside expressions and passed directly as arguments.
- Composition can remove temporary variables but shouldn't make code difficult to read.
- Each inner expression is evaluated before its result is used by the outer expression.

ThinkCScpp 3.5: Adding new functions

- Functions package a named sequence of statements into a reusable operation.
- A function definition specifies its return type, name, parameters, and body.
- Decomposing code into functions reduces duplication.
- Function names should describe the operation being performed.
- `main` can call functions that were declared before the call.

ThinkCScpp 3.6: Definitions and uses

- A function must be declared or defined before the compiler encounters a call to it.
- A declaration describes the function's interface without providing its implementation.

ThinkCScpp 3.7: Programs with multiple functions

- Multiple functions divide a program into smaller units with separate responsibilities.
- Execution begins in `main` and moves into other functions when they're called.
- Functions can call other functions when their declarations are visible.

ThinkCScpp 3.8: Parameters and arguments

- Parameters are variables declared by a function, while arguments are values supplied by the caller.
- Arguments are matched to parameters by position and compatible type.
- Passing by value gives the function its own copy of the argument.

ThinkCScpp 3.9: Parameters and variables are local

- Parameters and variables declared inside a function are local to that function.
- Separate functions can use the same local name without sharing storage.
- Local variables stop existing when their scope ends.
- A function can't directly access another function's local variables.

ThinkCScpp 3.10: Functions with multiple parameters

- Functions can accept multiple parameters separated by commas.
- Each parameter requires its own declared type.
- Arguments must be supplied in the expected order.

ThinkCScpp 3.11: Functions with results

- A non-`void` function returns a value using a `return` statement.
- The returned value's type must be compatible with the declared return type.
- Returned results can be stored, printed, or used inside larger expressions.
- Every reachable path should return a value when the function isn't `void`.
- Returning a result often makes a function easier to reuse than printing internally.

ProgFund 6: Program Control Functions

- Program control functions organize execution by separating tasks into callable modules.
- Functions can receive data, perform processing, and return results.

ProgFund 6.1: Pseudocode Examples for Functions

- Function pseudocode describes parameters, processing steps, and returned values without C++ syntax.
- It helps define responsibilities before implementation.
- Calls in the main algorithm show how individual modules interact.

ProgFund 6.2: Hierarchy or Structure Chart

- A structure chart shows which functions call other functions.
- It represents program organization without describing every statement.
- Higher-level functions coordinate work performed by lower-level modules.
- The chart can expose modules that are too large or tightly connected.

ProgFund 6.3: Program Control Functions

- Control functions coordinate other functions rather than performing every task directly.

ProgFund 6.4: Void Data Type

- A `void` function performs an action without returning a value.
- `void` can also indicate that a function doesn't accept parameters in some declarations.
- A `void` function may use `return` without an expression to exit early.

ProgFund 6.5: Documentation and Making Source Code Readable

- Readable code uses meaningful names, consistent formatting, and focused functions.
- Comments should explain intent or unusual decisions rather than repeat the code.
- Excessive comments can become inaccurate when the implementation changes.
- Documentation should describe interfaces, assumptions, and important constraints.
- Consistent style makes errors and structural problems easier to notice.

RooksGuide 5: Output

- Output streams send formatted values to destinations such as the console.
- `std::cout` uses << to insert data into the standard output stream.

RooksGuide 6: Input

- `std::cin` uses >> to extract typed values from standard input.
- Failed input leaves the stream in an error state until it's handled.
- Whitespace-delimited extraction won't read an entire line containing spaces.
- `std::getline` is better suited to full-line text input.

RooksGuide 7: Arithmetic

- Arithmetic expressions follow C++ precedence and type-conversion rules.
- Integer and floating-point operands can produce different results.
- Parentheses clarify calculations and reduce reliance on remembered precedence.

RooksGuide 8: Comments

- Comments document intent and are ignored during compilation.
- Line comments begin with `//`, while block comments use `/*` and `*/`.
- Good names and structure should carry most of the explanation.
- Comments shouldn't preserve obsolete code that version control already stores.
- Misleading comments are worse than missing comments.

RooksGuide 9: Data Types and Conversion

- Data types define representation, range, precision, and supported operations.
- Conversions allow expressions to combine different types.
- Some conversions are safe, while others can lose information.

RooksGuide 9.1: Floating-point types

- `float`, `double`, and `long double` offer different minimum precision and range.
- Floating-point arithmetic is approximate and subject to rounding.
- `double` is generally the default type for decimal literals.
- More precision doesn't make every decimal value exact.

RooksGuide 9.2: Other types introduced by C++11

- C++11 added features such as fixed-width integers, `nullptr`, and improved type inference.
- Fixed-width integer types are useful when an exact bit width is required.

RooksGuide 9.3: Conversion Between Types

- Implicit conversions occur when C++ automatically changes a value's type.
- Promotions usually preserve information, while narrowing conversions might not.
- Mixed-type expressions convert operands to a common type.
- Conversion behaviour should be considered before assigning the result.

RooksGuide 9.4: Coercion & Casting

- Coercion is an automatic conversion performed by the language.
- Casting explicitly requests that a value be treated as another type.
- `static_cast` clearly expresses many ordinary compile-time conversions.

RooksGuide 9.5: Automatic Types in C++11

- `auto` asks the compiler to infer a variable's type from its initializer.
- The inferred type remains static and can't later change like a JavaScript variable.
- `auto` can reduce repetition when the initializer already makes the type clear.
- It shouldn't hide information that readers need to understand the code.
- An `auto` variable normally requires an initializer.

3.3 Practice problems

x

3.4 Program design

3.5 Sources and references

1. cppreference.com. (n.d.). C++ reference. Retrieved Jan 2026 - Aug 2026, from https://cppreference.com/cpp
2. Braunschweig, D., & Busbee, K. L. (2018). Programming fundamentals: A modular structured approach (2nd ed.). Rebus Community. https://press.rebus.community/programmingfundamentals/
3. Hansen, J. A. (2013). The Rook's guide to C++. Rook's Guide Press. https://rooksguide.org/
4. Downey, A. B. (1999). Think C++ (Version 1.1.0). Green Tea Press. https://www.greenteapress.com/thinkcpp/

CHAPTER

FOUR

ASSIGNMENT TWO JOURNAL

CHAPTER

FIVE

ASSIGNMENT THREE JOURNAL

CHAPTER

SIX

ASSIGNMENT FOUR JOURNAL

CHAPTER

SEVEN

PRACTICE PROBLEMS

CHAPTER

EIGHT

THINK C++

I wrapped up this book in December 2025. It was an easy read, but I took down some of the most interesting "refresher" type notes. It's probably useful if you understand another language.

Total pages	191
Currently on	191

8.1 Chapter synopsis and notes

8.1.1 The way of the program

This chapter introduces what programming is and how C++ fits as a formal, compiled language. It defines programs as precise instructions, explains why errors are unavoidable, and frames debugging as a systematic, experimental activity. It contrasts formal and natural languages to set expectations about precision and walks through a minimal first program to establish structure.

- Programming demands precision; computers do exactly what is written.
- Debugging is a core activity, not a failure state.
- Compile-time, run-time, and logic errors are fundamentally different.
- Effective debugging is experimental and methodical.
- Formal languages eliminate ambiguity.

8.1.2 Variables and types

This chapter explains how programs store and manipulate data. It covers values, variables, assignment, output, keywords, operators, and precedence rules, including how characters are treated numerically. The core idea is building complexity by composing simple expressions under strict rules.

- Assignment changes state; it is not mathematical equality.
- Types constrain behavior and catch errors early.
- Operator precedence can silently change results.
- Characters are numeric under the hood.
- Expression composition is foundational.

8.1.3 Function

This chapter introduces functions as the primary tool for decomposition. It covers floating-point behavior, type conversion, math functions, defining and using functions, parameters, locality, and returning results to control data flow in multi-function programs.

- Functions manage complexity more than they save typing.
- Floating-point arithmetic is approximate.
- Parameters and local variables are scope-limited.
- Returning values is the main data flow mechanism.
- Programs are built from many small functions.

8.1.4 Conditionals and recursion

This chapter focuses on control flow through conditionals and recursion. It explains selective execution, chaining and nesting, the role of *return*, and how recursion works via the call stack, including common failure modes.

- Control flow determines execution paths.
- *return* exits a function immediately.
- Recursion depends on correct base cases.
- Infinite recursion usually means a missing stop condition.
- Stack diagrams clarify recursive execution.

8.1.5 Fruitful functions

This chapter deepens the use of functions that return values. It emphasizes incremental development, composition, overloading, boolean logic, and more advanced recursion patterns that rely on trusting correct function design.

- Functions with return values are easier to test and reuse.
- Boolean logic underpins decision-making.
- Overloading can reduce clarity if misused.
- Correct recursion relies on trusting contracts.
- Programs evolve incrementally.

8.1.6 Iteration

This chapter introduces repetition with loops. It covers multiple assignment, *while* and *for* loops, common looping patterns, nested iteration, and how encapsulation and generalization turn repeated logic into reusable functions.

- Loops express controlled repetition.
- *while* expresses logic; *for* expresses counting.
- Nested loops increase complexity quickly.
- Encapsulation turns patterns into tools.
- Generalization extends solutions beyond examples.

8.1.7 Strings and things

This chapter treats strings as containers of characters. It covers storage, indexing, length, traversal, searching, counting, concatenation, mutability, comparison, and character classification, emphasizing safe and repeatable patterns.

- Strings must be traversed carefully.
- Indexing errors are common bugs.
- Searching and counting follow standard patterns.
- Strings are mutable in C++.
- Character utilities avoid hard-coded logic.

8.1.8 Structures

This chapter introduces *struct* for grouping related data. It explains defining structures, accessing fields, passing them to functions, call-by-value vs call-by-reference, returning structures, and basic input handling.

- Structures model related data.
- Passing by reference enables efficiency and mutation.
- Returning structures is safe and common.
- Value vs reference affects behavior.
- Input handling requires defensive coding.

8.1.9 More structures

This chapter refines structured programming by focusing on design choices. It introduces pure functions, *const* correctness, modifiers, incremental development, and thinking in terms of reusable algorithms.

- Pure functions simplify reasoning.
- *const* communicates intent and safety.
- Modifiers change state; pure functions do not.
- Incremental development reduces risk.
- Algorithms are reusable problem-solving ideas.

8.1.10 Vectors

This chapter introduces vectors as dynamic sequences. It covers access, copying, iteration, size management, randomness, statistics, counting, histograms, and efficient single-pass algorithms.

- Vectors manage dynamic storage safely.
- Size must always be queried, never assumed.
- Single-pass algorithms are usually preferable.
- Randomness requires explicit seeding.
- Histograms reveal data distribution.

8.1.11 Member functions

This chapter moves behavior closer to data by introducing member functions. It explains implicit access to instance variables, constructors, header files, and separating interface from implementation.

- Behavior belongs with the data it operates on.
- Member functions implicitly know their object.
- Constructors enforce valid initial state.
- Header files define interfaces.
- Structure matters as programs grow.

8.1.12 Vectors of Objects

This chapter combines vectors with user-defined types. Using cards and decks, it introduces comparison, searching, sub-collections, and bisection search for efficiency.

- Collections of objects need comparison logic.
- Equality and ordering must be explicit.
- Search efficiency depends on organization.
- Sorted data enables faster algorithms.
- Decomposition applies to data structures.

8.1.13 Objects of Vectors

This chapter focuses on objects that contain vectors. It introduces enums, *switch*, shuffling, sorting, dealing, and mergesort, emphasizing composition and algorithmic thinking.

- Composition scales better than flat designs.
- Enumerations clarify state.
- Sorting and shuffling expose tradeoffs.
- Mergesort shows divide-and-conquer.
- Algorithms depend on structure, not content.

8.1.14 Classes and invariants

This chapter formalizes classes as a safer evolution of structures. It covers private data, accessors, invariants, preconditions, and encapsulation, using complex numbers as an example.

- Classes protect correctness.
- Private data enforces invariants.
- Accessors expose intent safely.
- Preconditions document assumptions.
- Invariants define valid states.

8.1.15 File Input/Output and apmatrixes

This chapter introduces streams and file I/O. It covers parsing text and numbers, simple data structures, matrices, and validating structured data from external sources.

- Files introduce persistence and failure modes.
- Parsing must be defensive.
- Data structures often mirror file layout.
- Matrices formalize 2D data.
- Correctness outweighs performance in I/O.

8.1.16 Appendix: AP class reference

Provides reference material for *apstring*, *apvector*, and *apmatrix*, used throughout the book to simplify learning.

- AP classes simplify learning.
- They abstract away STL complexity.
- Used consistently throughout the book.

CHAPTER

NINE

ROOKS GUIDE TO C++

I took a short break as work got busy and returned to finish this book in March 2026.

This book was shorter, but somehow felt a bit longer. I think because I read Think C++ first, then this one didn't feel as enjoyable as it had repeat content. I should have read them side by side, but instead went one before the other.

The content in the later half of this book was more enjoyable than the content in the first half, especially the arithmetic and dynamic data.

Total pages	160
Currently on	160

9.1 Chapter synopsis and notes

9.1.1 History

Sets context for why C++ exists and what problems it was built to solve, plus the rough evolution from C to "C with classes" to modern C++. The point is to understand the tradeoffs C++ keeps: performance, control over memory, and a large language surface area.

- C++ keeps low-level control and high-level abstraction in the same language.
- A lot of "weirdness" is historical baggage, not design elegance.
- You'll see multiple ways to do the same thing because the language evolved over decades.

9.1.2 Variables

Introduces naming values so programs can keep state. Focus is on declaration, initialization, scope basics, and how the compiler uses types to reason about what operations are allowed.

- Prefer initializing at declaration to avoid garbage state.
- Name things for meaning, not for the type.
- Most beginner bugs are "wrong variable, wrong scope, wrong value".

9.1.3 Literals and constants

Covers literal values (numbers, chars, strings) and making values non-changeable using constants. The goal is to reduce accidental mutation and encode assumptions in code.

- Constants make intent explicit and prevent accidental reassignment.
- Literal types matter (e.g., integer vs floating literal) because conversions can change results.

- Use constants to replace “magic numbers”.

9.1.4 Assignments

Explains how variables get new values using assignment, and how assignment differs from equality. Also covers compound assignment and common assignment patterns.

- Assignment mutates state; equality compares state.
- Compound assignments (+=, -=, etc.) are shorthand but still mutation.
- Watch for accidental assignment in conditions in languages that allow it.

9.1.5 Output

Introduces writing to the console and basic formatting. Emphasis is on building confidence by observing program state and results.

- Output is a debugging tool, not just user-facing UI.
- Formatting matters when humans must read the results.
- Separate “compute” from “print” when code starts to grow.

9.1.6 Input

Introduces reading from the user and dealing with the messiness of real input. Focus is on parsing, validation, and handling failure states.

- Input is adversarial by default; validate.
- Distinguish between reading tokens vs full lines.
- Always handle stream failure and leftover newline issues.

9.1.7 Arithmetic

Covers numeric operators and the difference between integer arithmetic and floating-point arithmetic. Includes precedence and common pitfalls (division, truncation).

- Integer division truncates; this is a frequent bug source.
- Modulus is for integers and is useful for digit extraction and periodic behavior.
- Overflow exists; don’t assume “big enough”.

9.1.8 Comments

Explains using comments to communicate intent, constraints, and non-obvious decisions. The goal is maintainability, not narrating every line.

- Comment “why”, not “what”.
- Outdated comments are worse than no comments.
- Prefer expressive code; comments are a supplement.

9.1.9 Data types and conversion

Covers built-in types, signedness, ranges, and converting between types. Emphasis is on loss of information, implicit conversions, and when to be explicit.

- Conversions can be narrowing (lose data) without being obvious.

- Signed/unsigned mixing creates surprising comparisons.
- Be explicit when precision or range matters.

9.1.10 Conditionals

Introduces branching (*if*, *else if*, *else*) and boolean logic. Focus is on writing correct conditions and avoiding deeply nested logic.

- Most bugs are wrong conditions, not wrong bodies.
- Prefer clear boolean expressions over cleverness.
- Short-circuit logic matters for safety (guard checks).

9.1.11 Strings

Covers strings as sequences of characters and common operations (length, indexing, concatenation, comparison). Emphasis is on boundary safety and input interaction.

- Indexing is where most string bugs live (off-by-one, bounds).
- Choose token input vs line input deliberately.
- Comparing strings is semantic; comparing pointers is not.

9.1.12 Loops

Introduces repetition (*while*, *do-while*, *for*) and loop control. Focus is on invariants, termination conditions, and common iteration patterns.

- Always know what makes the loop stop.
- Off-by-one errors are the default failure mode.
- Prefer *for* when counting and *while* when condition-driven.

9.1.13 Arrays

Introduces fixed-size contiguous storage and indexing. Emphasis is on bounds, lifetime, and why arrays are error-prone compared to safer containers.

- Arrays don't track their own size.
- Bounds violations can silently corrupt memory.
- Use safer containers when available, but understand arrays for fundamentals.

9.1.14 Blocks, functions, and scope

Explains *{}* blocks, variable lifetime, function decomposition, parameters, return values, and scope rules. Focus is on controlling visibility and reducing complexity.

- Scope is a tool to prevent misuse, not a nuisance.
- Functions should do one job and have clear inputs/outputs.
- Returning values beats global mutation for testability.

9.1.15 Problem solving & troubleshooting

Gives a workflow for breaking problems down and debugging systematically: reproduce, isolate, test assumptions, and verify fixes. Focus is on process over guessing.

- Reduce the problem until it's obvious.
- Add observability (prints, assertions) to test hypotheses.
- Fix the cause, not the symptom.

9.1.16 The preprocessor

Introduces *#include*, *#define*, and conditional compilation. Emphasis is on understanding that preprocessing happens before compilation and can create hard-to-debug behavior.

- Macros are text substitution, not typed functions.
- Prefer constants and inline functions over macros when possible.
- Header inclusion order and guards matter.

9.1.17 Advanced arithmetic

Expands into topics like numeric limits, precision, rounding, overflow behavior, and bitwise operations. The goal is to reason about correctness when numbers get tricky.

- Floating-point equality is usually wrong; compare with tolerance when needed.
- Bitwise ops are common in systems-level tasks, but easy to misuse.
- Know the numeric limits of your types.

9.1.18 File I/O

Introduces reading and writing files, stream state, and parsing structured text. Emphasis is on error handling and data validation.

- Always handle "file didn't open" and partial reads.
- Parsing needs a plan: format assumptions, separators, failure modes.
- Keep I/O separate from computation for cleaner code.

9.1.19 Pointers

Introduces addresses, dereferencing, and pointer semantics. Focus is on when indirection is needed, and how misuse leads to crashes or corruption.

- Uninitialized and dangling pointers are catastrophic.
- Pointer arithmetic is powerful and dangerous.
- Prefer references and safer abstractions unless pointers are required.

9.1.20 Dynamic data

Covers allocating data at runtime and managing lifetime. Emphasis is on ownership, leaks, and avoiding manual memory management where possible.

- Leaks happen when ownership isn't clear.
- Double-free and use-after-free are common failure modes.

- Prefer RAII-style ownership patterns rather than raw *new*/*delete*.

9.1.21 Classes and abstraction

Introduces building types that combine data and behavior with encapsulation. Focus is on invariants, interfaces, constructors, and keeping representations private.

- A class should protect its invariants by construction.
- Public API should be minimal and coherent.
- Encapsulation is about controlling mutation and coupling.

9.1.22 Separate compilation

Explains splitting programs into multiple files, headers vs source, declarations vs definitions, and linking. Focus is on build structure and avoiding multiple-definition errors.

- Headers declare; source files define.
- Include guards (or equivalents) prevent redefinition.
- Linker errors are usually "declared but not defined" or "defined twice".

9.1.23 STL

Introduces standard library containers, algorithms, and iterators as the idiomatic way to write safer and more expressive C++. Focus is on choosing the right container and leveraging algorithms.

- Prefer *std::vector*, *std::string*, and standard algorithms by default.
- Algorithms reduce bugs compared to hand-rolled loops.
- Iterators unify containers but can be confusing without practice.

CHAPTER

TEN

PROGRAMMING FUNDAMENTALS

Total pages	342
Currently on	0

10.1 Synopsis

10.2 Interesting notes

10.3 Definitions